

TinyGPT Text Generation — Experimental Report

1. Overview

This project trains a miniature GPT-style decoder-only transformer (**TinyGPT**) from scratch and measures how architectural and data changes affect training loss, throughput, and generated text quality.

The model is intentionally small — ~0.5 million parameters — so that training completes in minutes on a laptop GPU and the effects of each change are clearly visible.

Model Architecture

TinyGPT is a standard decoder-only transformer:

- Token embedding + learned positional embedding
- N stacked transformer blocks, each containing:
 - Multi-head causal self-attention with pre-layer norm
 - Position-wise feed-forward network (ReLU, single hidden layer)
- Final layer norm + linear head to vocabulary

Fixed Hyperparameters

Parameter	Value
d_model (embedding dimension)	128
n_head (attention heads)	4

d_ff (feed-forward hidden dim)	512
dropout	0.1
batch size	32
learning rate	3e-4 (AdamW)
epochs	200
optimizer gradient clip	1.0
generation temperature	0.8

Dataset

Two public-domain texts from Project Gutenberg:

Text	Size	Used in
<i>Romeo and Juliet</i> by Shakespeare	~150,000 chars	Baseline, Change 2, Change 3
<i>A Study in Scarlet</i> by Arthur Conan Doyle	~500,000 chars	Change 1

Characters are tokenized with a simple ASCII-level mapping (printable chars + newline + tab + 4 special tokens = vocab size 104).

2. Experiments

Four experiments are compared. Only one variable changes per experiment so the effect is isolated.

#	Name	Dataset	Context	Layers	Params
1	Baseline	Shakespeare	256	6	0.6M

2	Change: dataset	Sherlock Holmes	256	6	0.6M
3	Change: context	Shakespeare	128	6	0.6M
4	Change: layers	Shakespeare	256	4	0.4M

3. Results

3.1 Quantitative Comparison

Experiment	Final Loss	Train Time (s)	Tokens/s	GPU Mem
Baseline	1.1476	214.7	—	—
Change 1: dataset	1.1003	350.7	—	—
Change 2: context	0.6577	133.7	—	—
Change 3: layers	1.4031	145.6	—	—

3.2 Loss Curves

Each experiment produces a per-epoch loss plot showing the training curve over 200 epochs.

File: `output/Baseline_loss.png` — baseline Shakespeare training curve
File: `output/Change_1__dataset_loss.png` — Sherlock Holmes training curve
File: `output/Change_2__context_loss.png` — context=128 training curve
File: `output/Change_3__layers_loss.png` — 4-layer training curve

All four curves share a common shape: a steep drop in the first 20

epochs (loss $\sim 4.5 \rightarrow \sim 1.5$) followed by a gradual plateau. The plots differ in *where* they plateau and *how fast* they get there.

4. Experiment-by-Experiment Analysis

4.1 Baseline (Shakespeare, context=256, 6 layers)

Loss: 1.1476 | Time: 214.7s

The reference point. Loss drops exponentially and plateaus near 1.15. The model learns surface-level patterns of Shakespearean text: character names in ALL CAPS, line breaks, stage directions, and English-like word tokens — but it cannot produce coherent sentences or meaningful narrative.

Generated samples (temp=0.8):

Prompt: "ROMEO:"

→ "And ?JULIET. And blet by to no a stardvery pan a lover
There rud she pore in of a masticklade."

Prompt: "The king"

→ "the the Capulet. CAPULET. Hat that! So?s Sone inf I ha
JULIET. Mant nord face thou a frien your untl..."

Prompt: "It was a dark"

→ "e hou douse. JULIET. My will be I as saill weet whing
distest him. ROMEO. Sun the she shall in and speries."

Prompt: "Once upon a time"

→ "st coulent ard Capulety. FRIAR LAR LAWRENCE. Remat hen
Parif his. ROMEO. Fhather fall her of frin he I have l

Prompt: "The meaning of life is"

→ "thou voust ounse. POTERYBIO. No, be wille, fing bin I

that ond of the earm. PRINCE. But that me, sir..."

Observations:

- Character names (ROMEO, JULIET, CAPULET, PRINCE) appear correctly
- Stage directions ([_Exeunt._]) are present
- Text length and line breaks mimic the source
- Words are English-like but not real — the model learned *character-level bigrams and trigrams* without *meaning*
- This is expected: 0.6M parameters is $\sim 300,000\times$ smaller than GPT-3

What the model learned:

1. Token co-occurrence statistics (which chars follow which)
2. Formatting structure (ALL CAPS names, blank lines between speakers)
3. A vocabulary that resembles English spelling patterns

What it did not learn:

1. Grammar (subject-verb-object, tense, number agreement)
 2. Semantics (words have no meaning, only positional frequency)
 3. Narrative arc (no plot, no character consistency)
-

4.2 Change 1: Dataset (Sherlock Holmes, context=256, 6 layers)

Loss: 1.1003 | Time: 350.7s

What this tests: Does changing the training corpus change what the model outputs?

The Sherlock Holmes corpus is $\sim 3\times$ larger than Shakespeare ($\sim 500K$ chars vs $\sim 150K$). Each epoch processes more sequences, so training takes longer (350.7s vs 214.7s). Loss is slightly lower (1.100 vs 1.148) — more data per epoch makes the next-token prediction task marginally

easier.

The critical finding is visible in the generated text:

Prompt: "ROMEO:"

→ "CH. INCER II FONGregson for I well pecipe with JOFer.
IN WATLONON ANNT I. THER CHAPTELOPTER RIER V."

Prompt: "The king"

→ "was for the was have tall he fortun little le caud it
candly now him.? ?Well came on the compts..."

Prompt: "Once upon a time"

→ "upon of the constable which had eards of a leantling i
the camp of the had not count to him..."

Key observation: The vocabulary shifted completely. Baseline output contained "ROMEO", "JULIET", "CAPULET", "FRIAR". Sherlock output contains "Gregson", "Drebber", "constable", "JOFer" (Jefferson Hope?). The model is not learning "language" — it is memorizing the statistical distribution of its specific training text. The same architecture, trained on a different book, produces text that mimics that book's vocabulary and formatting.

Conclusion: The training data determines the domain of the output. The model is a *distributional mirror* of its corpus.

4.3 Change 2: Context Window (128 vs 256, Shakespeare)

Loss: 0.6577 | Time: 133.7s

What this tests: Does a shorter context window improve or degrade the model?

This produced the **lowest loss** and **fastest training** of all four experiments. The loss (0.66) is nearly half the baseline (1.15).

Why? With context=128, the sliding window generates roughly twice as

many training sequences from the same text. More importantly, the task is easier: predicting the next token from 128 previous tokens is less demanding than from 256. The model only needs to track local patterns.

Generated samples:

```
Prompt: "ROMEO:"  
→ "Onee! O madurself love me, on that we the hate. Good n  
me so me love me, course, go ald not say onf With hi"  
  
Prompt: "The king"  
→ "of me not her brow, That now she make me, a will be to  
death, Prick and make to his a our home."  
  
Prompt: "It was a dark"  
→ "and manishers. All is in the will I send. O, she lay!  
well, have bary! ROMEO. I dead, ter hate h"
```

Critical insight: Lower loss did **not** produce better text. The output is just as nonsensical as the baseline. The model got better at short-range predictions (2-3 word chunks fit in 128 tokens) but lost the ability to track longer patterns (it cannot reference a character introduced 200 tokens ago).

This demonstrates a fundamental principle: **loss measures prediction accuracy, not generation quality**. A model can achieve excellent loss by exploiting short-range statistics while being completely unable to maintain coherence over longer passages.

Trade-off:

Context	Loss	Coherence	Train Time
128	Lower (0.66)	Only short-range	Faster (134s)
256	Higher (1.15)	Potentially longer-range	Slower (215s)

4.4 Change 3: Fewer Layers (4 vs 6, Shakespeare)

Loss: 1.4031 | Time: 145.6s

What this tests: Does reducing model depth hurt quality?

This produced the **worst loss** of all four experiments. Removing 2 transformer layers (33% reduction) increased loss by 22% (1.40 vs 1.15). The model has less representational capacity — fewer attention- and-feedforward stages to build hierarchical features.

Training is faster per epoch (145.6s vs 214.7s for 6 layers) because each forward pass does less computation. But the final quality is noticeably worse.

Generated samples:

Prompt: "ROMEO:"

→ "He with in this my cord sickis and buried. ROMEO. The,
is cout they burss? Not with it mank the a I love..."

Prompt: "The king"

→ ". [_Exit._] JULIET. O mis thy ould stelps well temy th
She thou not for thur worswee like swell..."

Prompt: "Once upon a time"

→ "Capuliet; PARIWRENCE. Fome that not I day tre?s Capent
have sime, tray to ho wh my mearight not upotch."

The output still contains character names and stage directions — even 4 layers capture surface formatting. But the "sentences" are the most garbled of all experiments. With fewer layers, the model cannot build the hierarchical representations needed to predict even short-range token patterns reliably.

Conclusion: More layers → more capacity → better loss (up to the point where overfitting or diminishing returns set in). For this dataset and vocabulary, 6 layers outperforms 4, and the DGX Spark demonstration (`dgx_demo.py`) extends this test to 8 and 12 layers.

5. Summary Table

Experiment	Variable	Loss	Time	Key Finding
Baseline	—	1.15	215s	Reference; learns formatting, not meaning
Change 1	Dataset → Sherlock	1.10	351s	Model mirrors its training corpus
Change 2	Context → 128	0.66	134s	Lower loss ≠ better text
Change 3	Layers → 4	1.40	146s	Fewer layers underfit

6. Key Takeaways

- Surface patterns before semantics.** TinyGPT learns character co-occurrence, vocabulary, and formatting long before it learns anything resembling syntax or meaning. At 0.6M parameters, it never reaches the "meaning" stage — it only gets as far as plausible word-like tokens.
- Loss is not a proxy for quality.** Change 2 had the lowest loss but did not produce better text. Loss measures next-token prediction accuracy on the training set; it says nothing about whether the output is coherent, grammatical, or meaningful.
- Model capacity matters.** Reducing layers from 6 to 4 (Change 3) caused the largest loss increase. Depth directly controls the model's ability to build hierarchical representations.
- Data determines domain.** The same architecture trained on Shakespeare vs. Sherlock produces text that mimics each author's vocabulary and style. The model is a statistical mirror of its training distribution.

5. **Scaling up.** The DGX Spark demonstration (`dgx_demo.py`) extends these findings to larger models (20M params, 12 layers, 512 context) to show that with sufficient capacity and compute, the model produces noticeably more coherent text.